

Tìm kiếm nhị phân trên đáp án

Trong chapter trước bạn đã học binary search trên mảng - tìm vị trí của giá trị. Bài học này giới thiệu kỹ thuật nâng cao và quan trọng nhất liên quan tới binary search: tìm kiếm nhị phân trên đáp án (parametric search).

Đây là kỹ thuật giải hàng trăm bài toán tối ưu (min / max), miễn là kiểm tra được "đáp án có khả thi không" trong thời gian hợp lý.

1. Ý tưởng cốt lõi

Nhiều bài toán có dạng: "Tìm giá trị x nhỏ nhất (hoặc lớn nhất) sao cho một điều kiện $P(x)$ đúng".

Nếu P có tính đơn điệu tức là tồn tại một ngưỡng x_0 sao cho:

- Với $x \geq x_0$: $P(x)$ đúng
- Với $x < x_0$: $P(x)$ sai

(hoặc ngược lại) – thì ta có thể binary search trên giá trị x , mỗi bước chỉ cần kiểm tra $P(mid)$.

Mẫu code chuẩn (tìm x nhỏ nhất thỏa P)

C++

```
int lo = L0, hi = HI; // [L0, HI] chưa đáp án
while (lo < hi) {
    int mid = (lo + hi) / 2;
    if (check(mid)) hi = mid;           // Thỏa -> thu nhỏ
    else lo = mid + 1;                  // Không thỏa -> tăng lên
}
// lo == hi == đáp án
```

Python

```
lo, hi = L0, HI
while lo < hi:
    mid = (lo + hi) // 2
    if check(mid):
        hi = mid
    else:
        lo = mid + 1
# lo == hi == đáp án
```

Mẫu tìm x lớn nhất thỏa P

C++

```
int lo = L0, hi = HI;
while (lo < hi) {
    int mid = (lo + hi + 1) / 2; // (+1 de tranh vong lap vo han!)
    if (check(mid)) lo = mid;
    else hi = mid - 1;
}
```

Python

```
while lo < hi:
    mid = (lo + hi + 1) // 2 # +1 de tranh vong lap vo han
    if check(mid):
        lo = mid
    else:
        hi = mid - 1
```

Bẫy quen thuộc: Quên +1 trong mid $\rightarrow$ vòng lặp vô hạn khi $lo = mid$.

2. Bài toán mẫu 1: Chia gói hàng

Đề: Có n gói hàng, gói thứ i nặng w_i . Có k người chuyển hàng. Mỗi người được nhận một dãy liên tiếp các gói. Hỏi: chia thế nào để tổng nặng nhất một người phải vác là nhỏ nhất?

Phân tích:

- Đáp án nằm trong $[\max w_i, \sum w_i]$.
- Với X cố định, kiểm tra: "có chia được sao cho không ai vác $> X$ không? greedy: gộp liên tiếp đến khi tổng vượt X , sang người mới.
- Nếu chia được với $X \rightarrow$ cũng chia được với mọi $X' > X$. Tính đơn điệu OK!

C++

```
bool can(long long X) {
    int people = 1;
    long long load = 0;
    for (int i = 1; i <= n; i++) {
        if (w[i] > X) return false;
        if (load + w[i] > X) { people++; load = w[i]; }
        else load += w[i];
    }
    return people <= k;
}

long long lo = *max_element(w + 1, w + n + 1);
long long hi = accumulate(w + 1, w + n + 1, 0LL);
while (lo < hi) {
    long long mid = (lo + hi) / 2;
    if (can(mid)) hi = mid;
}
```

```

        else lo = mid + 1;
    }
    cout << lo;

```

Python

```

def can(X):
    people, load = 1, 0
    for wi in w:
        if wi > X:
            return False
        if load + wi > X:
            people += 1
            load = wi
        else:
            load += wi
    return people <= k

lo, hi = max(w), sum(w)
while lo < hi:
    mid = (lo + hi) // 2
    if can(mid):
        hi = mid
    else:
        lo = mid + 1
print(lo)

```

Độ phức tạp: $O(n \log W)$ với $W = \sum w_i$.

3. Bài toán mẫu 2: Cây bán kẹo (Aggressive Cows)

Đề: Có n vị trí trên trục số (đã sắp xếp). Đặt k con bò vào k trong n vị trí. Tìm cách đặt sao cho khoảng cách giữa hai con bò gần nhất là lớn nhất.

Phân tích:

- Đáp án D - kiểm tra: "có đặt được k con bò sao cho mỗi cặp cách nhau $\geq D$?" Greedy: đặt ở vị trí đầu, đi từ trái qua, đặt khi khoảng cách $\geq D$.
- Nếu D khả thi $\rightarrow$ mọi $D' < D$ cũng khả thi. Đơn điệu (ngược chiều) $\rightarrow$ tìm D lớn nhất thỏa.

C++

```

auto canPlace = [&](long long D) {
    int placed = 1;
    long long last = pos[1]; // Dat con dau o pos[1]
    for (int i = 2; i <= n; i++) {
        if (pos[i] - last >= D) {
            placed++;
            last = pos[i];
        }
    }
}

```

```

    }
    return placed >= k;
};

long long lo = 1, hi = pos[n] - pos[1];
while (lo < hi) {
    long long mid = (lo + hi + 1) / 2; // Tim max -> +1
    if (canPlace(mid)) lo = mid;
    else hi = mid - 1;
}
cout << lo;

```

Python

```

def can_place(D):
    placed, last = 1, pos[0]
    for i in range(1, n):
        if pos[i] - last >= D:
            placed += 1
            last = pos[i]
    return placed >= k

lo, hi = 1, pos[-1] - pos[0]
while lo < hi:
    mid = (lo + hi + 1) // 2
    if can_place(mid):
        lo = mid
    else:
        hi = mid - 1
print(lo)

```

4. Khi check chậm – cẩn thận với độ phức tạp

Nếu $\text{check}(\text{mid})$ là $O(n)$ thì tổng $O(n \log V)$ - thường rất nhanh.

Nếu $\text{check}(\text{mid})$ là $O(n^2) \rightarrow$ có thể TLE. Hãy luôn ước lượng tổng.

5. Binary search trên số thực (real binary search)

Khi đáp án là số thực, dùng vòng lặp cố định (thường 100-200 lần) thay vì so sánh $\text{lo} < \text{hi}$:

C++

```

double lo = L0, hi = HI;
for (int it = 0; it < 100; it++) {
    double mid = (lo + hi) / 2;
    if (check(mid)) hi = mid;
    else lo = mid;
}

```

```
cout << fixed << setprecision(6) << lo;
```

Python

```
lo, hi = L0, HI
for _ in range(100):
    mid = (lo + hi) / 2
    if check(mid):
        hi = mid
    else:
        lo = mid
print(f"{lo:.6f}")
```

100 lần lặp $\rightarrow$ sai số $\leq (hi - lo)/2^{100}$ – siêu nhỏ.

6. Khi nào áp dụng được?

Hãy đặt câu hỏi: "Nếu tôi đoán đáp án là X , kiểm tra có khả thi không có dễ không?" $\rightarrow$ search on answer là lựa chọn tuyệt vời!

Các dấu hiệu

- "Tìm giá trị nhỏ nhất sao cho ..."
- "Tối thiểu hóa giá trị tối đa" / "Tối đa hóa giá trị tối thiểu"
- "Có k ... sao cho ..."
- Thử ngược: nếu đề bài hỏi "tìm X " mà ta không biết cách, nhưng "với X cho trước, kiểm tra dễ" $\rightarrow$ binary search.

7. Bẫy thường gặp

- Off-by-one trong tính mid: tìm max $\rightarrow (lo + hi + 1)/2$.
- Vòng lặp vô hạn: do mid không thay đổi.
- Phạm vi $[lo, hi]$ sai: phải bao trùm đáp án.
- Tràn số: $(lo + hi)$ có thể tràn $\rightarrow$ dùng $lo + (hi - lo)/2$.
- Hiểu sai tính đơn điệu: nếu không đơn điệu, BS không áp dụng được!

8. Tổng kết

Binary search on answer là một trong những kỹ thuật quan trọng nhất ở mức Bạc. Nó biến bài toán tối ưu thành bài toán kiểm tra – thường dễ hơn rất nhiều.

Khi gặp bài tối ưu, hãy luôn tự hỏi: "Có thể kiểm tra đáp án không?" Nếu có, thử BS trước!